BCSL657D: DEVOPS LABORATORY MANUAL

VI Semester, B.E. (Computer Science & Engineering)

🛠️ PREREQUISITES & DEVELOPMENT ENVIRONMENT SETUP

WSL (Windows Subsystem for Linux) can be prone to path issues, outdated apt repositories, and permission errors. Follow this structured workflow to configure a clean, modern development environment.

1. Update Ubuntu Packages

Before running any installation scripts, update your packages:

```
sudo apt update && sudo apt upgrade -y
```

2. Install SDKMAN! (The Safest Way to Manage Java, Maven, & Gradle)

Instead of dealing with manual environment variable configuration or broken Ubuntu apt packages, use **SDKMAN!**. It automates path setup and manages versions perfectly.

```
# Install curl and zip if not present
sudo apt install curl zip unzip -y

# Download and install SDKMAN!
curl -s "[https://get.sdkman.io](https://get.sdkman.io)" | bash

# Load SDKMAN into your current terminal session
source "$HOME/.sdkman/bin/sdkman-init.sh"
```

Verify that SDKMAN! is installed:

```
sdk version
```

3. Install Java Development Kit (OpenJDK 11)

Use SDKMAN! to install a stable, modern JDK.

```
sdk install java 11.0.22-tem
```

Verify install:

```
java -version
```

4. Install Apache Maven

```
sdk install maven 3.9.6
```

Verify install:

```
mvn -version
```

5. Install Gradle

```
sdk install gradle 8.5
```

Verify install:

```
gradle -v
```

6. Install Ansible

Ansible is agentless and is easily fetched from Ubuntu's repository:

```
sudo apt install ansible -y
```

Verify install:

```
ansible --version
```

📂 EXPERIMENT 1: Introduction to Maven and Gradle

Overview of Build Automation Tools, Key Differences, Installation, and Setup.

Concept

- **Build Automation**: The process of scripting or automating the compilation of computer source code, packaging it into binary code (like JAR or WAR files), and running automated tests.
- **Maven**: A build tool based on the concept of a **Project Object Model (POM)** configured via XML (`pom.xml`). It relies heavily on strict conventions.
- **Gradle**: A modern build system that uses a Domain-Specific Language (DSL) based on **Groovy** or **Kotlin** instead of XML. It is task-based, highly customizable, and significantly faster due to incremental builds.

Deliverables

Demonstrate functional installations of Java, Maven, and Gradle in the terminal:

```
# Command 1: Java environment verification
java -version

# Command 2: Maven configuration verification
mvn -version

# Command 3: Gradle system verification
gradle -v
```

📂 EXPERIMENT 2: Working with Maven

Creating a Maven Project, Understanding the POM File, Dependency Management & Plugins.

Step 1: Generate a Standard Maven Project

Navigate to your working directory (such as `~/.local`) and generate a project from the quickstart template without interactive prompts:

```
mkdir -p ~/.local && cd ~/.local

mvn archetype:generate \
  -DgroupId=com.example \
  -DartifactId=demo \
```

```
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.4 \
  -DinteractiveMode=false
```

Step 2: Fix the Java 1.7 Compiler Target Bug

By default, the Quickstart archetype configures compilation settings for Java 1.7, which modern JDKs reject with compilation failures.

Open `pom.xml` inside `~/.local/demo` and replace its `<properties>` and `<dependencies>` configurations with this updated layout containing **Java 11 settings** and the **JavaFaker** dependency:

```
<project xmlns="[http://maven.apache.org/POM/4.0.0](http://maven.apache.org/POM/4.0.0
         xmlns:xsi="[http://www.w3.org/2001/XMLSchema-instance](http://www.w3.org/200
         xsi:schemaLocation="[http://maven.apache.org/POM/4.0.0](http://maven.apache.
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>
    </properties>

    <dependencies>
        <!-- Unit Testing Framework -->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>

        <!-- Library Dependency for Data Generation -->
        <dependency>
            <groupId>com.github.javafaker</groupId>
            <artifactId>javafaker</artifactId>
            <version>1.0.2</version>
        </dependency>
    </dependencies>
</project>
```

Step 3: Write Custom Logic utilizing Dependencies

Edit the default main class file: `src/main/java/com/example/App.java`

```
package com.example;

import com.github.javafaker.Faker;

public class App {
    public static void main(String[] args) {
        Faker faker = new Faker();
        System.out.println("================================");
        System.out.println("  MAVEN DEMO ACTIVE");
        System.out.println("================================");
        System.out.println("Full Name: " + faker.name().fullName());
        System.out.println("Superhero Aligned: " + faker.superhero().name());
        System.out.println("Space Habitat: " + faker.space().planet());
        System.out.println("================================");
    }
}
```

Step 4: Compile and Run

Ensure your terminal is in the folder containing `pom.xml` and run:

```
mvn compile exec:java -Dexec.mainClass="com.example.App"
```

📁 EXPERIMENT 3: Setting Up a Gradle Project

Understanding Build Scripts, Dependency Management, and Task Automation.

Step 1: Initialize the Gradle Wrapper App

Create a separate project directory, navigate into it, and generate a Java application scaffold:

```
mkdir -p ~/gradle-demo && cd ~/gradle-demo
gradle init
```

Provide the following responses to the setup wizard:

1. Select type of build to generate: **1 (application)**
2. Select implementation language: **1 (Java)**
3. Enter target Java version: **11** (or 17)
4. Split functionality across multiple subprojects: **1 (no)**
5. Select build script DSL: **1 (Groovy)**

6. Generate build using new APIs: **no**
7. Select test framework: **1 (JUnit 4)**
8. Project name: **[Enter for default: gradle-demo]**
9. Source package: **com.example**

Step 2: Configure Dependencies and Custom Tasks

Open `app/build.gradle` and inspect its clean, declarative code block. Add the **JavaFaker** dependency and register a custom pipeline task:

```
plugins {
    id 'application'
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation 'junit:junit:4.13.2'
    implementation 'com.google.guava:guava:31.1-jre'

    // Add JavaFaker Library
    implementation 'com.github.javafaker:javafaker:1.0.2'
}

application {
    mainClass = 'com.example.App'
}

// Custom task registration demonstrating execution mechanics
tasks.register('helloBuddy') {
    doLast {
        println "---------------------------------"
        println "Automation Task 'helloBuddy' executed!"
        println "---------------------------------"
    }
}
```

Step 3: Run the Application and the Custom Task

Run using the Gradle wrapper (`./gradlew`):

```
# Execute application main class
./gradlew run

# Execute custom automation task
```

```
./gradlew helloBuddy
```

📂 EXPERIMENT 4: Build and Run with Maven, Migrate to Gradle

Build a Java App with Maven, package it, and migrate the identical project to Gradle.

Step 1: Create a Fresh Maven Application

```
cd ~
mvn archetype:generate -DgroupId=com.example -DartifactId=maven-to-gradle -Darchetype
```

Step 2: Configure Java Target in the Maven POM

Change directories to `maven-to-gradle`. Open `pom.xml` and replace its target properties to make it buildable on modern JDKs:

```
<properties>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
</properties>
```

Step 3: Package into a JAR

Execute the packaging lifecycles:

```
mvn package
```

This compiles, runs tests, and packages your binary into `target/maven-to-gradle-1.0-SNAPSHOT.jar`.

Step 4: Perform the Automated Gradle Migration

Instead of manual translation, tell Gradle's engine to read the POM structure and generate a corresponding build system:

```
gradle init
```

Prompts:

- Found a Maven build. Generate a Gradle build from it? **yes**
- Select build script DSL: **1 (Groovy)**
- Generate build using new APIs: **no**

Step 5: Test the Migrated App

Gradle translates the `pom.xml` structure into a clean `build.gradle` file. Compile and run your migrated application:

```
./gradlew run
```

📂 EXPERIMENT 5: Installing and Configuring Jenkins

Installing and configuring Jenkins for initial use.

Standard Issue

In WSL environments, running systemd commands (`sudo systemctl start jenkins`) is unstable and often fails. To ensure smooth performance, **always run Jenkins as a standalone web archive container**.

Step 1: Fetch Jenkins Server Binary

```
cd ~
wget [https://get.jenkins.io/war-stable/latest/jenkins.war](https://get.jenkins.io/wc
```

Step 2: Fire Up Jenkins

Launch Jenkins on the standard HTTP web port `8080` :

```
java -jar jenkins.war --httpPort=8080
```

Note: Keep this terminal window open! This process must run in the background.

Step 3: Capture the Setup Password

Look closely at your running console output logs. A boxed warning message will display containing your initial configuration credentials:

```
*************************************************************
*************************************************************
Jenkins initial setup is required. An admin user has been created...
Please use the following password to proceed to installation:

a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6

*************************************************************
*************************************************************
```

Copy this code!

Step 4: Complete Web UI Setup

1. Open your browser on Windows: `http://localhost:8080`
2. Paste the setup password and click **Continue**.
3. Select **"Install suggested plugins"** and allow the installer to run.
4. Create your first administrative user (e.g., Username: `admin`, Password: `password`). Click **Save and Finish**.

📂 EXPERIMENT 6: Continuous Integration with Jenkins

Automating a local Maven compile-and-package pipeline.

Step 1: Register Local System Maven with Jenkins

Jenkins needs to know where your command line Maven installation lives.

1. On your Jenkins Dashboard, click **Manage Jenkins** -> **Tools** (or *Global Tool Configuration*).
2. Scroll to the **Maven** configurations.
3. Click **Add Maven**.
4. Set Name: `Local-Maven`
5. Uncheck **"Install automatically"**.
6. Set **MAVEN_HOME**: `/home/madhavan/.sdkman/candidates/maven/current` (or your local system `MAVEN_HOME` pathway. You can find this in terminal by typing `mvn -version`).
7. Click **Save**.

Step 2: Create a Freestyle Build Job

1. Click **New Item** on the Jenkins home sidebar.
2. Item name: `My-First-CI-Pipeline`

3. Select **Freestyle project** and click **OK**.
4. In the configuration window, scroll down to the **Build Steps** block:
    - Click **Add build step** and choose **Invoke top-level Maven targets**.
    - Maven Version: Select `Local-Maven`.
    - Goals: `clean package`
5. Click the **Advanced...** button below the Goals input field.
    - Locate the **POM** text box.
    - Provide the absolute path to your Experiment 2 manual demo application:

```
/home/madhavan/.local/demo/pom.xml
```

6. Click **Save** at the bottom of the page.

Step 3: Run the Automation Job

1. In the project sidebar, click **Build Now**.
2. Look at the Build History tracking widget. Click on Build `#1` and choose **Console Output**.
3. Watch Jenkins successfully fetch the project blueprint, run its packaging steps, and print `FINISHED: SUCCESS`.

📂 EXPERIMENT 7: Configuration Management with Ansible

Automated configuration using a YAML Playbook to configure an NGINX Web Server.

Step 1: Create a Local Inventory Host File

Ansible tracks target devices using inventory files. We will configure Ansible to run commands on our local WSL terminal natively.

1. Create an isolated lab environment folder:

```
mkdir -p ~/ansible-lab && cd ~/ansible-lab
```

2. Create an inventory hosts index:

```
nano hosts
```

3. Add the local host configuration variables:

```
[local]
localhost ansible_connection=local
```

*(Save and exit using **Ctrl+O**, **Enter**, **Ctrl+X**).*

Step 2: Ping Test the Inventory

Verify that Ansible can communicate with the local host configuration:

```
ansible -i hosts local -m ping
```

Expected Green Response:

```
localhost | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

Step 3: Write the NGINX Installation Playbook

1. Create a playbook configuration file:

```
nano install_nginx.yml
```

2. Paste the declarative automation steps inside:

```
---
- name: Automate NGINX Web Server Installation
  hosts: local
  become: true
  tasks:
    - name: Ensure the package cache is updated
      apt:
        update_cache: yes

    - name: Install NGINX Web Server package
      apt:
        name: nginx
        state: present
```

```
    - name: Ensure NGINX service is started and active
      service:
        name: nginx
        state: started
```

(Save and exit).

Step 4: Validate and Run your Infrastructure Script

1. Test your YAML syntax configuration structure:

```
ansible-playbook -i hosts install_nginx.yml --syntax-check
```

2. Execute the deployment. Use the `--ask-become-pass` flag so Ansible can securely request your root/sudo password to install NGINX:

```
ansible-playbook -i hosts install_nginx.yml --ask-become-pass
```

Step 5: Verify Deployment

Verify that NGINX was installed and is running on your network port:

```
curl http://localhost
```

You will see HTML code on your screen ending with `<h1>Welcome to nginx!</h1>`.

📂 EXPERIMENT 8: Jenkins CI and Ansible Deployment Pipeline

Linking Maven, Jenkins, and Ansible to create a functional CI/CD pipeline.

The Goal

Jenkins will trigger a build, use Maven to generate a `.jar` package, and then invoke Ansible to automatically copy and deploy that `.jar` artifact.

Step 1: Write an Ansible Deployment Playbook

Create a script that takes the packaged build file generated by Jenkins, moves it to a production deployment folder, and executes it.

1. Open your terminal and create a playbook file inside your Ansible workspace:

```
nano /home/madhavan/ansible-lab/deploy_lab8.yml
```

2. Paste this configuration:

```
---
- name: Deploy Lab 8 Artifact from Jenkins
  hosts: local
  tasks:
    - name: Create secure deployment folder
      file:
        path: /home/madhavan/production_app
        state: directory
        mode: '0755'

    - name: Copy JAR file from Maven workspace to production folder
      copy:
        src: /home/madhavan/.local/demo/target/demo-1.0-SNAPSHOT.jar
        dest: /home/madhavan/production_app/app.jar
        mode: '0755'

    - name: Run the deployed Java application
      command: java -jar /home/madhavan/production_app/app.jar
      register: application_output

    - name: Display runtime output in Jenkins console
      debug:
        msg: "{{ application_output.stdout_lines }}"
```

(Save and exit).

Step 2: Create the Pipeline inside the Jenkins Dashboard

1. Go to your Jenkins Dashboard browser page: `http://localhost:8080` .
2. Click **New Item** on the sidebar.
3. Name: `Lab8_DevOps_Pipeline`
4. Choose **Freestyle project** and click **OK**.

Step 3: Link Maven and Ansible Tasks in Jenkins

Scroll down to the **Build Steps** container box.

Task A: Build with Maven (Continuous Integration)

1. Click **Add build step** and choose **Invoke top-level Maven targets**.
2. Maven Version: Select your configured `Local-Maven`.
3. Goals: `clean package`
4. Click **Advanced...**
5. POM Path: `/home/madhavan/.local/demo/pom.xml`

Task B: Deploy with Ansible (Continuous Deployment)

1. Click the **Add build step** dropdown button *again* (directly underneath the Maven step) and select **Execute shell**.
2. In the text commands box, paste the path to execute your Ansible playbook:

```
ansible-playbook -i /home/madhavan/ansible-lab/hosts /home/madhavan/ansible-lab/c
```

3. Click **Save** at the bottom of the page.

Step 4: Run the Complete CI/CD Pipeline

1. On your new project workspace page, click **Build Now**.
2. Open Build `#1` and select **Console Output** to monitor the execution logs.
3. Check the output stream. You will see Jenkins package your application using Maven, invoke your Ansible playbook shell command, deploy the `.jar` to your production path, run the app, and output:

```
TASK [Display runtime output in Jenkins console] ******************************
ok: [localhost] => {
    "msg": [
        "================================",
        "  MAVEN DEMO ACTIVE",
        "================================",
        "Full Name: [Random Name]",
        "Superhero Aligned: [Random Hero]",
        "Space Habitat: [Random Planet]",
        "================================"
    ]
}
```